

A Population Based Approach for ACO

Michael Guntsch¹ and Martin Middendorf²

¹ Institute for Applied Computer Science and Formal Description Methods
University of Karlsruhe, Germany
`guntsch@aifb.uni-karlsruhe.de`

² Computer Science Group
Catholic University of Eichstätt-Ingolstadt, Germany
`martin.middendorf@ku-eichstaett.de`

Abstract. A population based ACO (Ant Colony Optimization) algorithm is proposed where (nearly) all pheromone information corresponds to solutions that are members of the actual population. Advantages of the population based approach are that it seems promising for solving dynamic optimization problems, its finite state space and the chances it offers for designing new metaheuristics. We compare the behavior of the new approach to the standard ACO approach for several instances of the TSP and the QAP problem. The results show that the new approach is competitive.

1 Introduction

The ability of real ants to find short paths between their nest and food sources is based on indirect communication by the use of pheromones. This behavior of ant colonies has motivated the design of the Ant Colony Optimization (ACO) metaheuristic (Dorigo et al. [1,2]). The ACO metaheuristic has been applied to several combinatorial optimization problems for which the construction of a solution can be described as a path in a decision graph (see [3] for an introduction and overview).

An ACO algorithm is an iterative search process where in every iteration (generation) each of m (artificial) ants constructs one solution by following a path through the decision graph. For each decision on which edge in the decision graph the ant should follow it uses pheromone information which stems from former ants that have found good solutions. As an example consider a simple ACO algorithm for the Traveling Salesperson problem (TSP). The pheromone information is stored in a so called pheromone matrix where element τ_{ij} , indicates how good it seems to move from city i to city j , for $i, j \in [1 : n] = \{1, \dots, n\}$. The probabilistic decisions of the ants are made according to a local decision rule where the probabilities of the possible outcomes are correlated to the relative amounts of pheromone on the corresponding edges in the decision graph. In addition an ant may also use heuristic information for its decisions. Thus, the pheromone information and the heuristic information are indicators of how good it seems to follow the corresponding edge.

In this paper a modification of the way pheromone information is updated is proposed and tested on the Traveling Salesperson Problem (TSP) and the Quadratic Assignment Problem (QAP). To limit the amount of pheromone in the matrix and gradually make old updates weaker, the standard approach is to multiply all values τ_{ij} with a factor $(1 - \rho)$, $0 < \rho < 1$ after every iteration before any positive reinforcement is done on good solutions. The method we propose is to keep track of all good solutions up to a certain age and thereafter remove their influence explicitly from the pheromone matrix through a negative update on the τ_{ij} values they had modified. Hence, the value with which an element of the pheromone matrix was initialized is at the same time it's minimum value, since negative updates occur only to cancel out previous positive updates of the same magnitude. The effect of having a minimum pheromone value has been applied successfully for ACO algorithms by Stützle and Hoos [4,5].

One motivation for introducing a population based scheme is the planned application to dynamic optimization problems. An ACO approach for a dynamic TSP problem where instances change at certain intervals through the deletion and insertion of cities has been studied in [6,7]. Strategies for pheromone modification in reaction to changes of the problem instance together with an elitist strategy for use in dynamic environments have been proposed. A standard elitist strategy for ACO algorithms is that an elitist ant updates the pheromone values in every generation according to the best solution found so far. But after a change of the problem instance the best solution found so far will usually no longer represent a valid solution to the new instance. Instead of simply forgetting the old best solution, it was adapted so that it becomes a reasonably good solution for the new instance. Clearly, this is only reasonable when the changes of the instance are not too strong.

Since there exist often good strategies to adapt solutions for an old instance to a new instance we propose here to maintain in addition to a possible elitist solution a population of other solutions. As members of this population we chose the last k solutions that have been used to update the pheromone matrix. Since in dynamic environments pheromone that stems from very old instances is usually less valuable than pheromone of ants that worked on recent instances we propose to skip all the very old pheromone. That means we skip all pheromone corresponding to solutions that are not in the actual population. In other words, every pheromone value depends only on solutions that are in the actual population and for every solution in the population there is some amount of pheromone on the corresponding elements in the pheromone matrix.

Often in dynamic real time environments it is important to have an algorithm that can react to changes quickly. Therefore we propose a fast pheromone update process. Instead of applying evaporation to every pheromone value the amount of pheromone that corresponds to a solution in the population is not changed. Only when a solution leaves the population its amount of pheromone is subtracted from the corresponding pheromone values.

Since after each generation of ants one solution enters the population and one solution leaves the population the pheromone update is done only for pheromone

values corresponding to this two solutions. In case of an n city TSP this means that the number of pheromone values that have to be changed is at most $2n$ for the new method (using one addition or subtraction operation for each value) compared to n^2 for most versions of the standard method (using one multiplication operation for each value).

It has to be mentioned that Maniezzo [8] proposed a method for pheromone update that is also fast. In his method an ant that found a solution that is better than the average solution quality of the k former ants adds a certain amount of pheromone to the corresponding pheromone values. Otherwise, if the solution is worse than this average, pheromone is subtracted from the corresponding pheromone values. In case of the TSP this means that for every ant n additions respectively subtractions are done (plus a constant number of arithmetic operation to determine the actual amount of pheromone that is added/subtracted).

To ensure that every pheromone value has a minimum value we do not remove the initial amount of pheromone (cmp. [4,5]). Otherwise it could happen that pheromone values become zero because there is no solution in the population that puts pheromone on them. But then the ants cannot make the corresponding decision.

Another motivation for introducing a population based approach is that it might allow a different approach to handle it theoretically due to the finite state space. Moreover, a population based approach offers interesting possibilities for designing new metaheuristics.

In this study we show that the population based approach performs at least not worse than the standard method on static problems. Hence, there are good chances that it is useful in particular for dynamic problems and for using it in new metaheuristics. The population based approach is tested experimentally on several instances of the TSP and the Quadratic Assignment problem (QAP). Moreover it is investigated how the size of the population influences the optimization behavior.

The paper is organized as follows. In Section 2 we describe a standard ACO algorithm and the new population based approach. The test setup is given in Section 3. The experimental results are discussed in Section 4, and conclusions are drawn in Section 5.

2 ACO Approaches

In this section we describe ACO algorithms that work according to the standard approach and according to the new population based approach for the TSP and the QAP problem. It should be mentioned that the aim of this study is not to find the best ACO algorithm for our test problems but to compare the basic approaches. It is clear that the algorithms for both approaches can be improved. For an example the ants can change the pheromone values during their search for a solution ([9]) or local search can be applied to the solutions found by the ants (see [10,11,5]).

2.1 Standard Approach

The ACO algorithm that uses the standard approach for the TSP uses ideas from Dorigo et al. [2,9]. In every generation each of m ants constructs one tour through all the given n cities. Starting at a random city an ant selects the next city using heuristic information as well as pheromone information. For each pair (i, j) , $i, j \in [1 : n]$, $i \neq j$ of cities there is heuristic information, denoted by η_{ij} , and pheromone information, denoted by τ_{ij} . The heuristic value is $\eta_{ij} = 1/d_{ij}$ where d_{ij} is the distance between city i and city j .

With probability q_0 , where $0 \leq q_0 < 1$ is a parameter of the algorithm, an ant at city i chooses next city j from the set S of cities that have not been visited so far which maximizes $\tau_{ij}^\alpha \cdot \eta_{ij}^\beta$, where α and β are constants that determine the relative influence of the heuristic values and the pheromone values on the decision of the ant. With probability $1 - q_0$ the next city is chosen according to the probability distribution over S determined by

$$p_{ij} = \frac{\tau_{ij}^\alpha \cdot \eta_{ij}^\beta}{\sum_{h \in S} \tau_{ih}^\alpha \cdot \eta_{ih}^\beta} \quad (1)$$

Before doing the pheromone update some of the old pheromone is evaporated for all i, j according to

$$\tau_{ij} \mapsto (1 - \rho) \cdot \tau_{ij}$$

where parameter $\rho \in (0, 1)$ determines the evaporation rate. For pheromone update an elitist strategy is used where one elitist ant updates pheromone along the best solution found so far, i.e. for every city i some amount of pheromone is added to element (i, j) of the pheromone matrix when j is the successor of i in the so far best found tour. A weight $0 \leq w_e \leq 1$ determines the relative amount of pheromone placed by the elitist ant compared to the amount placed by the best ant in the current generation (the extreme cases are $w_e = 0$ and $w_e = 1$ where no respectively all of the new pheromone is from the elitist ant). For initialization we set $\tau_{ij} = \tau_{init}$ where $\tau_{init} > 0$ for every edge (i, j) .

For the QAP problem there are n facilities, n locations, and $n \times n$ matrices $A = [a_{ij}]$ and $B = [b_{ij}]$ where a_{ij} is the distance between locations i and j and b_{hk} is the flow between facilities h and k . The problem is to find an assignment of facilities to locations, i.e., a permutation f of $[1 : n]$ such that the following measure is minimized $\sum_{i=1}^n \sum_{j=1}^n a_{f(i)f(j)} b_{ij}$. The ACO algorithm for QAP uses pheromone values τ_{ij} , $i, j \in [1 : n]$ which refer to the assignment of facility i to location j . No heuristic is used for the QAP.

Max-Min version. Since the pheromone values in the population based approach are never zero we use also a version of the standard ACO algorithm where the pheromone values that are used by the ants for their decisions have a minimal value. This version of the algorithm is called the Max-Min ACO algorithm (Note, that the idea to use minimum values stems from the MAX-MIN ACO algorithm proposed in [4,5]). The Max-Min ACO algorithm used the same pheromone matrix as the standard ACO algorithm. Instead of using the pheromone values τ_{ij}

directly to determine the probability of a decision as in (1) every ant of the Max-Min ACO algorithm determines the probability using the values $\tau_{ij} + \tau_+ \cdot \tau_{init}$ where $\tau_+ > 0$ is a parameter. Hence, the minimal value is $\tau_+ \cdot \tau_{init}$ and (1) is replaced by

$$p_{ij} = \frac{(\tau_{ij} + \tau_+ \cdot \tau_{init})^\alpha \cdot \eta_{ij}^\beta}{\sum_{h \in S} (\tau_{ih} + \tau_+ \cdot \tau_{init})^\alpha \cdot \eta_{ih}^\beta} \quad (2)$$

2.2 Population Based Approach

In the population based approach the first generation of ants works in the same way as in the standard approach, i.e., the ants search solutions using the initial pheromone matrix and the best ant in the generation adds pheromone to the pheromone matrix. But no pheromone evaporation is done. The best solution is then put in the (initially empty) solution population. After k generations there are exactly k solutions in the population. From generation $k + 1$ on one solution in the population is removed from the population and the corresponding amount of pheromone is subtracted from the elements of the pheromone matrix. That means for all pheromone values corresponding to the removed solution the same amount of pheromone is subtracted that was added when this solution had entered the population. In this paper we chose to remove the oldest solution from the population. We call this algorithm FIFO-Queue ACO algorithm. Observe that pheromone values are never zero since the initial amount of pheromone remains. A parameter τ_{max} determines the maximum amount of pheromone. The amount of pheromone that an ant updates is then $(1 - w_e)(\tau_{max} - \tau_{init})/k$.

3 Test Setup

We evaluated the standard, the Max-Min and the FIFO-Queue ACO algorithms on the Traveling Salesperson Problem (TSP) and the Quadratic Assignment Problem. The following test-instances (from TSPLIB [12] respectively QAPLIB [13]) were used for a comparison of the standard and the FIFO-Queue ACO algorithm:

eil101 , 101 cities, symmetric (euclidean)	wil50 , size 50, symmetric
kroA100 , 100 cities, symmetric (euclidean)	wil100 , size 100, symmetric
d198 , 198 cities, symmetric (euclidean)	tai100a , size 100, asymmetric
kro124p , 100 cities, asymmetric	tai100b , size 100, asymmetric
ftv170 , 170 cities, asymmetric	

We used parameters values $\rho = 0.1, 0.05, 0.02, 0.01, 0.005$ with a corresponding number of iterations $t = 2000, 5000, 10000, 20000, 50000$ for the standard ACO algorithm. The number of iterations was chosen to ensure that it had converged in (nearly) each of its runs. The FIFO-Queue ACO algorithm was evaluated with maximum pheromone values $\tau_{max} = 1.0, 3.0, 10.0$ ¹ and population

¹ For the symmetric TSPs, the actual maximum pheromone-value is $\tau_{max}/2$ because of symmetric updating.

sizes $k = 1, 5, 25$ and ran for $t = 50000$ iterations for all parameter combinations, since convergence is not possible. Furthermore, these parameter-settings were combined with $q_0 = 0.0, 0.5, 0.9$ and $w_e = 0.0, 0.25, 0.5, 0.75, 1.0$.

A (so far) limited comparison was done with the Max-Min ACO algorithm. All combinations of $\tau_+ \in \{0.1, 0.3, 1.0\}$, $\rho \in \{0.01, 0.05, 0.25\}$, and $w_e \in \{0.0, 0.25, 0.5, 0.75, 1.0\}$ were tested on the TSP instances **eil101**, **kroA100**, and **kro124p**, and on the QAP instances **ta100a** and **wil50**. Since neither type of algorithms converges to a point where no further improvement to the solution can be expected, only $t=50000$ was evaluated.

The other parameters for all three ACO algorithms were $m = 10$ ants per iteration, $\alpha = 1$, and $\beta = 5$, $\tau_{init} = 1/(n - 1)$ ($\beta = 0$, $\tau_{init} = 1/n$) for TSP (respectively QAP). Each parameter combination was tested with 10 different random seeds for the standard and FIFO-Queue ACO algorithm and 5 different random seeds for the Max-Min ACO algorithm and the average was taken.

4 Results

4.1 Comparison with Standard Approach

In order to determine whether the FIFO-Queue ACO algorithm is a competitive alternative to the standard one, we ranked the different parameter combinations (altogether 210, 75 for the standard and 135 for the FIFO-Queue ACO algorithm) according to their average performance on the individual problem instances. From these instance specific ranks, we took the averages for the two problem classes to generate one class-specific rank for each parameter combination. Of course, this method cannot be used for a detailed comparison of solution quality, but the statement “a better average rank implies at least as good as a performance on average” holds, enabling us to ascertain whether the FIFO-Queue ACO algorithm is competitive. Tables 1 and 2 show the average ranks of the best respective parameter combination for the standard and FIFO-Queue ACO algorithm after different number of iterations and for different values of ρ . The number of iterations used in the tables are $t_0 = t/100$, $t_1 = t/50$, $t_2 = t/25$, $t_3 = t/10$, $t_4 = t/5$, $t_5 = t/2.5$, and $t_6 = t$ for a total number of iterations t that the standard ACO algorithm ran.

For both tables, with the exception of using $\rho = 0.005$ at time t_5 for QAP, the best combination of parameters of the FIFO-Queue ACO algorithm always achieves a better average rank than the standard ACO algorithm for any t_i , indicating that its performance is indeed competitive.

The average rank (for all t_i) over all parameter combinations was 66.5 for TSP and 53.2 for QAP. The tables show that the average rank of the best parameter combination was significantly lower for both the standard and the FIFO-Queue ACO algorithm on QAP than on TSP. This seems to indicate that more of a distinction between good and bad parameter sets can be made for QAP, whereas for the TSP the different parameter combinations perform closer to one another in terms of quality. Also the use of a heuristic for TSP might result in

Table 1. Average ranks for standard and FIFO-Queue ACO algorithm on TSP

		t_0	t_1	t_2	t_3	t_4	t_5	t_6
$\rho = 0.1$	Standard	27.0	34.8	50.6	57.0	77.0	89.4	101.6
	Queue	9.0	17.8	16.8	19.4	20.2	16.2	14.0
$\rho = 0.05$	Standard	47.4	50.6	49.4	54.2	71.8	85.2	92.6
	Queue	18.8	18.0	19.6	19.2	16.6	14.4	11.6
$\rho = 0.02$	Standard	65.2	64.4	47.8	43.2	53.0	62.2	77.4
	Queue	17.8	19.8	19.0	17.0	15.2	11.8	13.6
$\rho = 0.01$	Standard	69.2	68.2	48.0	43.4	55.8	61.0	69.8
	Queue	18.6	18.4	16.0	15.4	11.6	14.0	14.6
$\rho = 0.005$	Standard	75.2	66.2	48.6	51.2	58.2	64.2	71.6
	Queue	17.2	15.8	14.8	12.2	13.6	15.0	18.8

Table 2. Average ranks for standard and FIFO-Queue ACO algorithm on QAP

		t_0	t_1	t_2	t_3	t_4	t_5	t_6
$\rho = 0.1$	Standard	27.4	25.8	14.4	4.6	2.4	6.6	25.8
	Queue	4.0	4.2	3.0	3.6	3.6	4.2	5.2
$\rho = 0.05$	Standard	38.2	26.2	17.0	7.0	5.0	8.6	27.4
	Queue	2.8	3.4	2.6	3.2	5.2	5.8	5.2
$\rho = 0.02$	Standard	40.8	41.6	28.2	15.8	8.0	7.0	17.0
	Queue	3.4	2.6	2.6	4.8	5.8	5.8	6.2
$\rho = 0.01$	Standard	46.8	44.6	31.8	19.6	8.2	7.0	18.0
	Queue	2.6	2.6	3.6	5.2	5.6	6.2	7.4
$\rho = 0.005$	Standard	44.8	41.8	31.2	14.0	9.0	3.0	14.8
	Queue	2.2	4.4	5.0	5.6	6.4	7.8	7.2

a more similar behavior of different parameter combinations. For TSP, the best parameter combination seems to be more instance-specific than for QAP.

Note that for any row in tables 1 and 2 the combination of parameters that is best does not necessarily stay the same. Instead, it undergoes a characteristic development depending on at what time t_i best performance was sought. Tables 3 and 4 give examples for the parameter-development of the best combinations that takes place in such a row. For the standard ACO algorithm, the typical development towards parameters that focus more on exploration than on exploitation when given ample time can be seen with declining q_0 and w_e , for TSP as well as QAP.

For the FIFO-Queue ACO algorithm, the situation is less clear. For TSP, all best combinations from Table 1 used population size $k = 1$, which means that the only guidance for the ants of a given generation are the solutions of the elite ant and of the best working ant from the previous iteration, and the heuristic. It seems that the ability to quickly explore the neighborhood of a promising solution (last iteration's best), combined with a drive toward the best solution found so

Table 3. Parameter-development over time for the best respective combinations on TSP (top 3 rows for standard ACO algorithm, bottom four for FIFO-Queue), $\rho = 0.01$.

$\rho = 0.01$	200	400	800	2000	4000	8000	20000
q_0	0.5	0.5	0.0	0.0	0.0	0.0	0.0
w_e	0.75	0.75	0.75	0.5	0.5	0.25	0.25
τ_{max}	1.0	1.0	1.0	1.0	1.0	1.0	1.0
q_0	0.5	0.5	0.5	0.5	0.5	0.5	0.5
w_e	0.25	0.25	0.25	0.25	0.25	0.25	0.25
τ_{max}	3.0	1.0	1.0	1.0	1.0	1.0	1.0
k	1	1	1	1	1	1	1

Table 4. Parameter-development over time for the best respective combinations on QAP (top 3 rows for standard ACO algorithm, bottom four for FIFO-Queue), $\rho = 0.01$.

$\rho = 0.01$	200	400	800	2000	4000	8000	20000
q_0	0.9	0.5	0.5	0.0	0.0	0.0	0.0
w_e	1.0	0.75	1.0	0.75	0.5	0.25	0.25
τ_{max}	1.0	1.0	1.0	1.0	1.0	1.0	1.0
q_0	0.5	0.9	0.9	0.9	0.9	0.9	0.9
w_e	0.75	0.5	0.5	0.5	0.5	0.75	0.75
τ_{max}	10.0	3.0	3.0	3.0	3.0	3.0	10.0
k	1	5	5	5	5	25	1

far and a strong heuristic influence to prevent choices that seem drastically bad, works best. Of the best five FIFO-Queue combinations for the scenario of Table 3, four used population size $k = 1$ and one $k = 5$. A positive elite weight $w_e \in \{0.25, 0.5, 0.75\}$ was also used by all these combinations, suggesting that some degree of enforcement of the best solution found so far is beneficial. These traits are combined with $q_0 = 0.5$ and at first $\tau_{max} = 3$, after t_1 $\tau_{max} = 1.0$. $q_0 = 0.9$ and $\tau_{max} = 1.0$ achieve a similar effect in the FIFO-Queue ACO algorithm, both determining how likely it is that an ant will choose the best (q_0) or one of the best (τ_{max}) choices according to pheromone and heuristic information. The medium q_0 value and low τ_{max} indicate that deviating often from the best path found so far produces good results.

For QAP, the parameter combinations which performed well are somewhat different from those for TSP. The only similarity is the strong guidance provided by an elite ant with $w_e \in \{0.5, 0.75\}$. However, both a higher q_0 and a higher τ_{max} value suggest that it is beneficial to stay closer to the best placement(s) encoded in the pheromone matrix. Also, the larger population size of $k = 5$ and once even $k = 25$ point out that a greater diversity in solutions encoded into the matrix is helpful. The probable reason for this is the absence of a heuristic to generate an uneven probability distribution.

4.2 Comparison with Max-Min ACO Algorithm

To compare the FIFO-Queue ACO algorithm with the Max-Min ACO algorithm use a rank-based comparison as in Subsection 4 (altogether 84 different parameter combinations, 45 for the Max-Min and 39 for the FIFO-Queue ACO algorithm).

Table 5. Average ranks for Max-Min and FIFO-Queue ACO algorithm on TSP

	500	1000	2000	5000	10000	20000	50000
Max-Min	20.6	14.6	21.6	14.0	22.3	17.3	17.0
Queue	2.6	4.6	11.6	11.3	10.6	11.3	12.6

Table 6. Average ranks for Max-Min and FIFO-Queue ACO algorithm on QAP

	500	1000	2000	5000	10000	20000	50000
Max-Min	18.5	17.5	14	13	11	8.5	8
Queue	12	13.5	13	9	4.5	7.5	10

Tables 5 and 6 show the average rank of the best parameter combination for FIFO-Queue and Max-Min ACO algorithm. The main observation that can be made is that for all cases but one (QAP after 50000 iterations), the average rank of the FIFO-Queue ACO algorithm is lower than that of the Max-Min ACO algorithm, indicating that the former's performance is at least as good as the latter's. This suggests that the solution quality which is achieved by the FIFO-Queue ACO algorithm is not only due to the implicit minimum and maximum pheromone values, but also to the quick adaptation a small explicit population of solutions is able to perform.

5 Conclusion

The aim of this paper was to establish a population based ACO algorithm as an alternative to standard ACO algorithms. The performance measured on the two problem classes (TSP and QAP) show that the new approach is competitive.

The new approach offers several possibilities for future work. First, it seems well suited for coping with dynamic optimization problems which require a fast re-orientation of the pheromone information. Second, due to the discrete nature of the population and hence the pheromone matrix of the FIFO-Queue ACO algorithm, it offers starting points for a theoretical analysis. Third the approach offers possibilities for new metaheuristics that use the population of solutions.

References

1. M. Dorigo. *Optimization, Learning and Natural Algorithms* (in Italian). PhD thesis, Dipartimento di Elettronica, Politecnico di Milano, Italy, 1992. pp. 140.
2. M. Dorigo, V. Maniezzo, and A. Coloni. The ant system: Optimization by a colony of cooperating agents. *IEEE Trans. Systems, Man, and Cybernetics – Part B*, 26:29–41, 1996.
3. M. Dorigo and G. Di Caro. The ant colony optimization meta-heuristic. In D. Corne, M. Dorigo, and F. Glover, editors, *New Ideas in Optimization*, pages 11–32. McGraw-Hill, 1999.
4. T. Stützle and H. Hoos. Improvements on the ant system: Introducing MAX(MIN) ant system. In *Proc. of the International Conf. on Artificial Neural Networks and Genetic Algorithms*, pages 245–249. Springer Verlag, 1997.
5. T. Stützle and H.H. Hoos. MAX-MIN ant system. *Future Generation Computer Systems Journal*, 16(8):889–914, 2000.
6. M. Guntsch and M. Middendorf. Pheromone modification strategies for ant algorithms applied to dynamic TSP. In E.J.W. Boers et al., editor, *Applications of Evolutionary Computing: Proceedings of EvoWorkshops 2001*, number 2037 in Lecture Notes in Computer Science, pages 213–222. Springer Verlag, 2000.
7. M. Guntsch, M. Middendorf, and H. Schmeck. An ant colony optimization approach to dynamic TSP. In L. Spector et al., editor, *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO-2001)*, pages 860–867. Morgan Kaufmann Publishers, 2001.
8. V. Maniezzo. Exact and approximate nondeterministic tree-search procedures for the quadratic assignment problem. *INFORMS Journal on Computing*, 11(4), 1999.
9. M. Dorigo and L.M. Gambardella. Ant colony system: A cooperative learning approach to the traveling salesman problem. *IEEE Transactions on Evolutionary Computation*, 1:53–66, 1997.
10. L.-M. Gambardella, E.D. Taillard, and M. Dorigo. Ant colonies for the quadratic assignment problem. *Journal of the Operational Research Society*, 50:167–76, 1999.
11. T. Stützle and M. Dorigo. ACO algorithms for the quadratic assignment problem. In D. Corne, M. Dorigo, and F. Glover, editors, *New Ideas in Optimization*, pages 33–50. McGraw-Hill, 1999.
12. <http://www.iwr.uni-heidelberg.de/groups/comopt/software/tsplib95>.
13. <http://www.opt.math.tu-graz.ac.at/qaplib>.